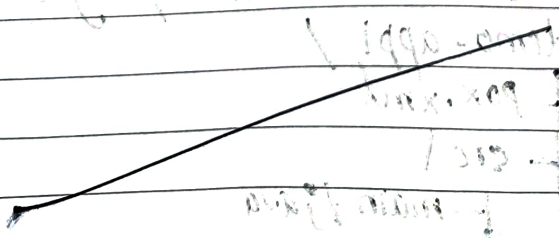
classmate
Date
Page

11-09-2026

create folder → VS code → create file (.txt)
Open terminal
git init
git add .
git commit -m "Initial commit"
git config --local user.name "Prarthana"
git config --local user.email "vvce24cse0308@vvce"
git config --list
git remote add origin "url of github"
git branch -M main
git push -u origin main
git branch (to check which branches are present)
git checkout -b branch (new branch created)
git switch main
(to switch b/w branches)
make changes in file.txt and save
git add .
git commit -m "updated commit"
git switch branch1
make changes in file.txt & save
git add .
git commit -m "updated in branch1"
git switch main
git merge branch1
git add .
git commit -m "updated in main branch"

git reset --soft / --hard



Experiment 2

25-09-2026

Create Maven project

Step 1: mvn archetype:generate -DgroupId=com.example.app -DartifactId=demo-app -DarchetypeArtifactId=maven-archetype-quickstart -DinteractiveMode=false

Step 1: Java has been already installed check the version

Step 2: Install Maven using below commend

CMD: sudo apt update
sudo apt install maven -y

Step 3: Create maven project
CMD: mvn archetype:generate -DgroupId=com.example.app -DartifactId=demo-app1 -DarchetypeArtifactId=maven-archetype-quickstart -DinteractiveMode=false

Step 4: Change the directory to demo-app using below command
CMD: cd demo-app1

Step 5: List all the files from the demo-app1 directory
CMD: dir

Step 6: Understand the project structure

```
demo-app1/
├── pox.xml
├── src/
│   ├── main/java
│   └── test/java
```

step 7: edit the pom.xml
CMD: notepad pom.xml

```
<project xmlns="http://maven.apache.org/POM/4.0.0"
  xmlns:xsi="http://www.w3.org/2001/XMLSchema-
  instance"
  xsi:schemaLocation="http://maven.apache.org/POM/
  4.0.0 http://mavan.apache.org/maven-v4_0_0.xsd">
<modelVersion>4.0.0</modelVersion>
<groupId>com.example.app</groupId>
<artifactId>demo-appl</artifactId>
  <packaging>jar</packaging>
  <version>1.4.14</version>
</dependency>

<!-- Logging -->
<dependency>
    <groupId>org.slf4j</groupId>
    <artifactId>slf4j-api</artifactId>
    <version>2.0.9</version>
</dependency>

<!-- Apache commons -->
<dependency>
    <groupId>org.apache.commons</groupId>
    <artifactId>commons-lang3</artifactId>
    <version>3.14.0</version>
</dependency>

<!-- Compiler Plugin -->
<plugin>
    <groupId>org.apache.maven.plugins</groupId>
    <artifactId>maven-compiler-plugin</artifactId>
    <version>3.11.0</version>
```

```
        <configuration>
            <source>17</source>
            <target>17</target>
        </configuration>
    </plugin>

    <!-- Test Plugin -->
    <plugin>
        <groupId>org.apache.maven.plugins</groupId>
        <artifactId>maven-surefire-plugin</artifactId>
        <version>3.1.2</version>
    </plugin>
  </plugins>
</build>
</project>
```

Step 8: Write the java application code.

CMD: notepad src/main/java/com/example/app/App.java

```
package com.example.app;

import org.slf4j.Logger;
import org.slf4j.LoggerFactory;
import org.apache.commons.lang3.StringUtils;

public class App
{
    private static final Logger logger = LoggerFactory.
                                    getLogger(App.class);

    public static void main(String[] args)
    {
        String name = "CI/CD pipeline";
```

```
if (StringUtils.isNotBlank(name))
{
    String message = greet(name);
    logger.info(message);
    System.out.println(message);
}
else
{
    logger.error("Name is empty");
}
}

public static String greet(String name)
{
    return "Hello, " + name + "! Welcome to Maven
        CI/CD Demo";
}
}
```

Step 9: Write test code

CMD: notepad src/test/java/com/example/app/AppTest.java

```
package com.example.app;
import org.junit.Test;
import static org.junit.Assert.assertEquals;
public class AppTest {
    @Test
    public void testGreet() {
        String result = App.greet("Student");
        assertEquals("Hello Student! Welcome to Maven
            CI/CD Demo", result);
    }
}
```

step 10: Build the project
CMD : mvn clean install

step 11 : Run the application
CMD : mvn exec:java -Dexec.mainClass="com.example.app.App"

Shaikh

Experiment 3

8/10/26

1. Create a folder → create dockerfile in word document → textfile in notepad

2. Inside dockerfile

```
FROM tomcat:10.1-jdk17
COPY index.html /usr/local/tomcat/webapps/ROOT/index.html
EXPOSE 8080
CMD ["catalina.sh", "run"]
```

3. Inside notepad create a html file

```
<!DOCTYPE html>
<html>
<head>
    <title> Docker tomcat applications </title>
</head>
<body>
    <h1> Welcome to Dockerized web application </h1>
    <p> This static web application is running inside an apache tomcat docker container </p>
    <h2> Application containerization using Docker </h2>
    <p> Succesfully deployed using Docker and Apache tomcat </p>
</body>
</html>
```

4. Convert Dockerfile.docx to Dockerfile and index.html.txt to index.html

```
ren Dockerfile.txt Dockerfile
ren index.html.txt index.html
```

Then dir.
Check the file whether they are renamed.

5 docker build -t tomcat-web-app10 .

This command is used to create a Docker image from your Dockerfile.

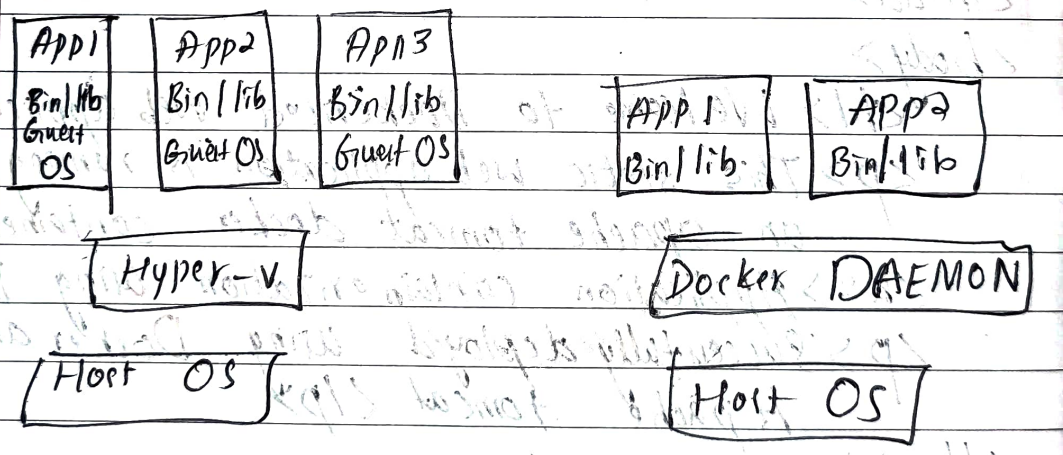
6 docker rm tomcat-web-app10

It is used to remove a Docker container.

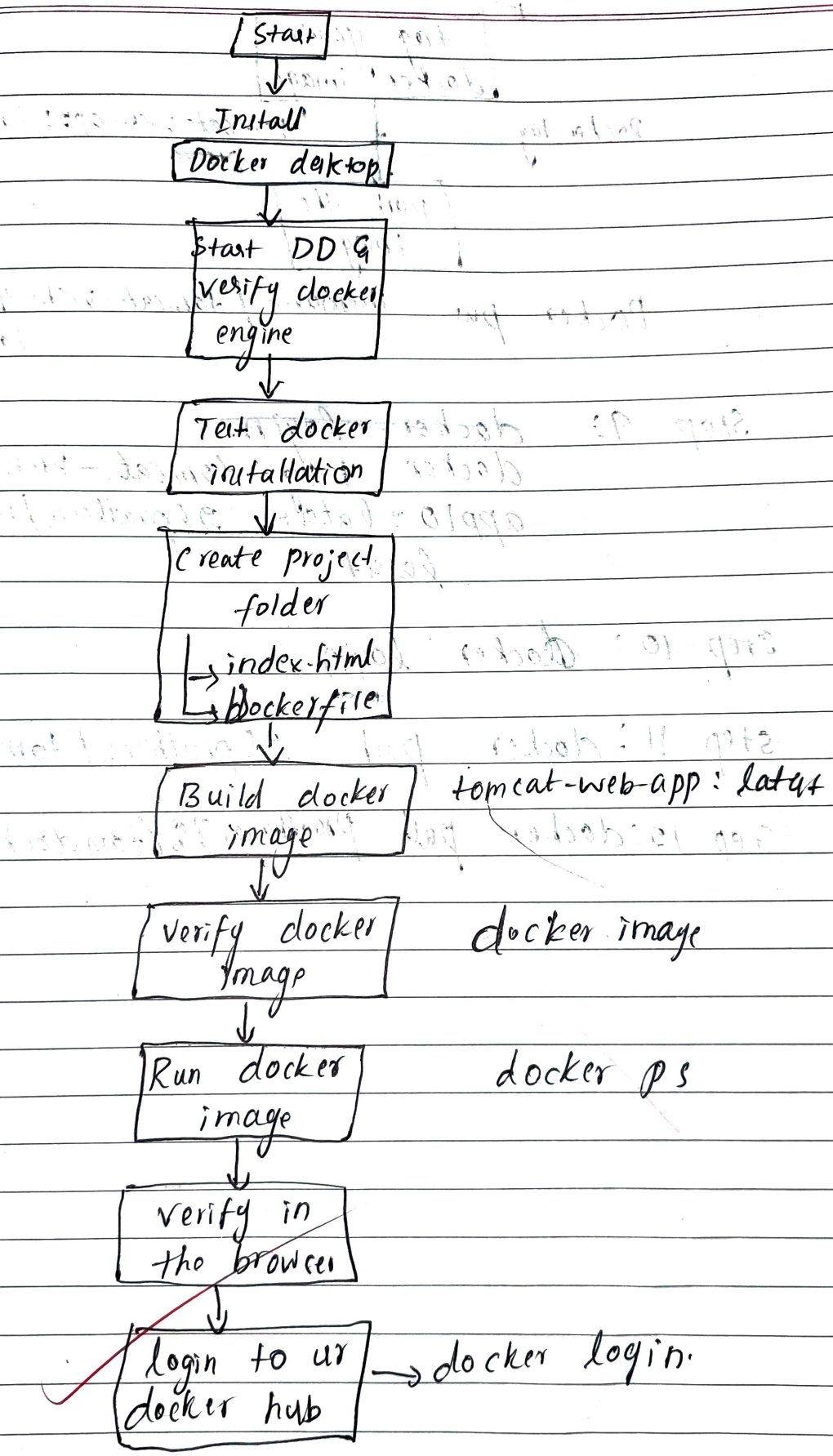
7 docker run -d -p 8081:8080 --name tomcat-web-app10 tomcat-web-app10

This is used to create & start a Docker container from your image

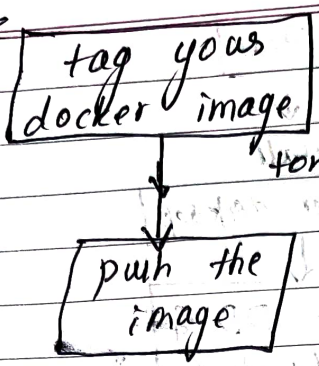
8 docker ps



H/W level virtualization


Start
Initall
Docker desktop
Start DD G
verify docker
engine
Test docker
intallation
Create project
folder
index.html
Dockerfile
Build docker
image
tomcat-web-app : latest
verify docker
image
docker image
Run docker
image
docker ps
verify in
the browser
login to ur
docker hub
docker login.

classmate
Date
Page



Docker push username/tomcat-web-app:latest

Step 9: ~~docker login.~~
docker tag tomcat-web-app10:latest 28prathana/tomcat:latest

step 10: docker login.

step 11: docker push 28prathana/tomcat:latest

Step 12: docker pull prarthana378/tomdocker:latest

classmate
Date
Page

K8s cluster

Control plane
- kubeapi server
- etcd
- kube scheduler
- control manager
- Cloud control manage

klocker nodes
- kubelet
- container runtime (containerd)

API server

* Entry points for kubernetes cluster and api requests
* Store k8s cluster state and configuration data
* kube scheduler : select a suitable node for newly scheduled pod.
* Control manager : Run controller that reconsile the actual state with desired state.
* Cloud api server : Integrate K8s with supported cloud infrastructure when applicable.
* kubelet : Ensure pods assigned to the nodes are running as specified.
* container runtime : Run container

9/10/26

Experiment 4

16/10/24

create cluster using kind 4 nodes → control plane

kubectl get nodes

→ notepad deployment.yaml
→ notepad service.yaml
→ kubectl apply -f deployment.yaml → kubectl get pods → 3 pods
→ kubectl apply -f service.yaml → kubectl get service
→ k8s
tomcat-service

kubectl port-forward service/tomcat-web-service 8081:8080

Open a new terminal.

Invoke-WebRequest http://localhost:8081
-UseBasicParsing

Open the browser & type the above url.

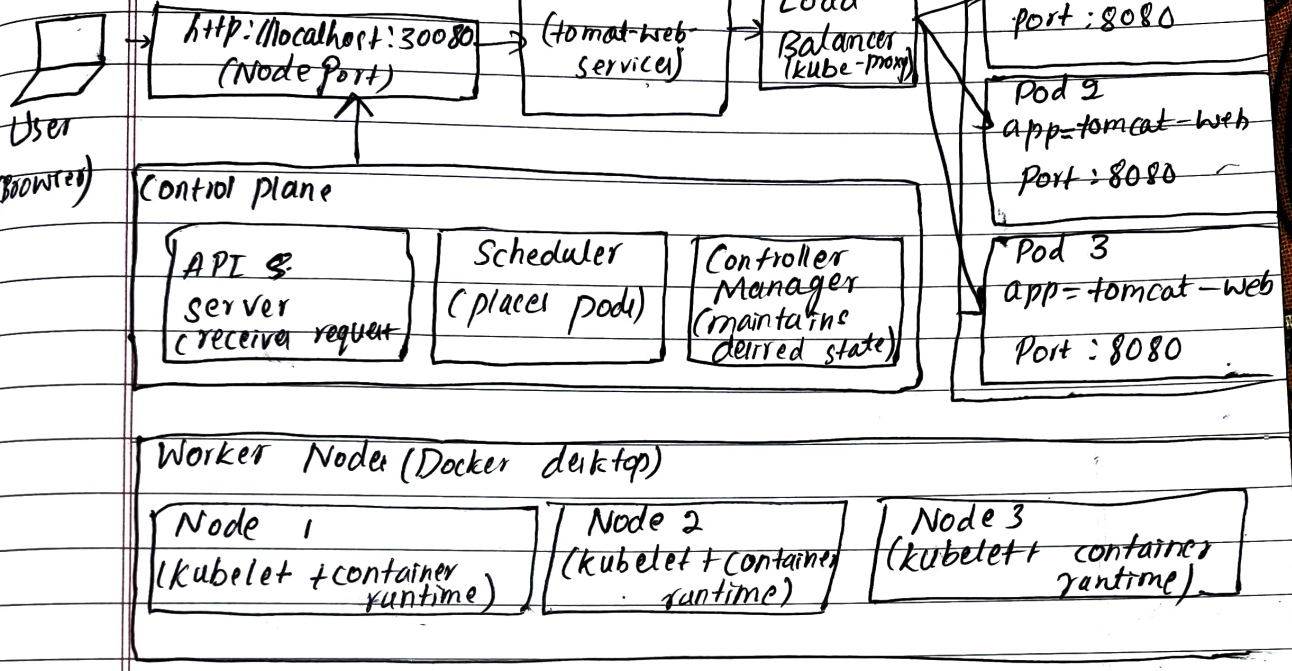
kubectl scale deployment tomcat-web-deployment --replicas=5

Kubectl get pods -W

kubectl delete pod tomcat-web-deployment-7f8f5d8745-fsdvb

kubectl get endpointslices




Pod (3 replica)
Pod 1
app=tomcat-web
port: 8080
Pod 2
app=tomcat-web
Port: 8080
Pod 3
app=tomcat-web
Port: 8080
User (Browser)
http://localhost:30080 (NodePort)
Service (tomcat-web service)
Load Balancer (kube-proxy)
Control plane
API Server (receive request)
Scheduler (place pod)
Controller Manager (maintain desired state)
Worker Node (Docker desktop)
Node 1 (kubelet + container runtime)
Node 2 (kubelet + container runtime)
Node 3 (kubelet + container runtime)